LHandProLib SDK 手册

> **版本：1.1**
> **作者：深圳市灵巧驱控技术有限公司**
> **目标平台：Windows/Linux，支持 EtherCAT/CAN/CANFD/RS485 通讯协议**

简介

`LHandProLib` 是一个用于控制灵巧手（Dexterous Hand）的 C++ SDK，支持多种通讯方式（EtherCAT、CAN、CANFD、RS485），提供对灵巧手的自由度控制、传感器数据读取、回零、状态监控等功能。本手册详细说明 SDK 中所有接口、枚举类型及使用示例。

命名空间

所有功能均定义在 `lhplib` 命名空间中：

```
namespace lhplib { ... }
```

错误码（LER）

错误码	宏定义	含义
`0`	`LER_NONE`	执行成功
`1`	`LER_PARAMETER`	参数错误
`2`	`LER_KEY_FUNC_UNINIT`	关键函数未初始化（如未设置回调）
`3`	`LER_GET_CONFIGURATION`	读取配置失败
`4`	`LER_COMM_CONNECT`	通讯连接错误
`5`	`LER_COMM_SEND`	通讯发送错误
`6`	`LER_COMM_RECV`	通讯接收错误
`7`	`LER_COMM_DATA_FORMAT`	通讯数据格式错误
`8`	`LER_INVALID_PATH`	无效的文件路径
`9`	`LER_LOG_SAVE_FAIL`	日志文件保存失败

错误码	宏定义	含义
999	LER_UNKNOWN	未知错误

自由度（DOF）枚举

枚举值	含义
LAC_DOF_6	6自由度（默认）
LAC_DOF_15	15自由度

通讯类型（LCN）枚举

枚举值	含义
LCN_ECAT	EtherCAT 通讯
LCN_CAN	CAN 通讯
LCN_CANFD	CANFD 通讯
LCN_RS485	RS485 通讯

控制模式（LCM）枚举

枚举值	含义
LCM_POSITION	位置控制
LCM_VELOCITY	速度控制
LCM_TORQUE	力矩控制
LCM_VEL_TOR	速度+力矩混合控制
LCM_POS_TOR	位置+力矩混合控制
LCM_HOME	回零模式

运行状态（LST）枚举

枚举值	含义
`LST_STOPPED`	正常停止状态
`LST_RUNNING`	正常运行状态
`LST_ALARM`	报警停止状态
`LST_POS_LIMIT`	正限位状态
`LST_NEG_LIMIT`	负限位状态
`LST_BOTH_LIMIT`	正负限位同时触发
`LST_EMG_STOP`	急停状态
`LST_HOMING`	回零运行状态

传感器ID（LSS）枚举

枚举值	含义
`LSS_FINGER_1_1`	大拇指指尖
`LSS_FINGER_1_2`	大拇指指腹
`LSS_FINGER_2_1`	食指指尖
`LSS_FINGER_2_2`	食指指腹
`LSS_FINGER_3_1`	中指指尖
`LSS_FINGER_3_2`	中指指腹
`LSS_FINGER_4_1`	无名指指尖
`LSS_FINGER_4_2`	无名指指腹
`LSS_FINGER_5_1`	小拇指指尖
`LSS_FINGER_5_2`	小拇指指腹
`LSS_HAND_PALM`	手掌传感器
`LSS_MAX_COUNT`	最大传感器数量（用于数组分配）

左右手方向（LDR）枚举

枚举值	含义
`LDR_HAND_RIGHT`	右手（默认）
`LDR_HAND_LEFT`	左手

函数指针类型

类型	描述
`LogAddCallback`	日志回调函数：`void(*)(const char*)`
`SendDataCallback`	CAN/RS485 发送回调：`bool(*)(unsigned int, const unsigned char*, unsigned int)`
`ECSendDataCallback`	EtherCAT RPDO 发送回调：`bool(*)(const unsigned char*, unsigned int)`

LHandProLib 类接口说明

构造与析构函数

`LHandProLib()`

构造函数，初始化内部资源和私有对象（Pimpl 模式）。

`~LHandProLib()`

析构函数，释放资源，自动调用 `close()`。

初始化与关闭

`int initial(int mode)`

初始化灵巧手驱动程序。

- **参数**：
  - `mode`：通讯类型，取值为 `LCN_ECAT` / `LCN_CAN` / `LCN_CANFD` / `LCN_RS485`
- **返回值**：
  - `0`：执行成功

    - 其他：参考错误码 `LER_*`
- **前置条件**：
    i. 底层通讯设备已连接
    ii. 已设置发送回调（`set_send_data_callback` 或 `set_send_rpdo_callback`）
    iii. 已设置接收数据处理（`set_recv_data_decode` 或 `set_tpdo_data_decode`）

`void close()`

关闭驱动程序，释放资源，断开所有连接。

通讯回调设置

`void set_send_data_callback(SendDataCallback callback)`

设置 CAN/RS485 数据发送回调。

- **参数**：
    - `callback`：函数指针，原型为 `bool(unsigned int id, const unsigned char* data, unsigned int len)`
        - `id`：CAN ID 或设备地址
        - `data`：待发送数据指针
        - `len`：数据长度（≤8）
        - 返回值：`true` 表示发送成功，`false` 表示失败
- **说明**：该回调将在 SDK 需要发送数据时被调用

`void set_send_data_callback_ex(void* callback_impl)`

设置 `std::function` 包装的发送回调（适用于 C++11 函数对象）。

- **参数**：
    - `callback_impl`：指向 `std::function<bool(unsigned int, const unsigned char*, unsigned int)>` 的指针
- **注意**：必须保证该 `std::function` 对象生命周期长于 `LHandProLib` 实例

`void set_recv_data_decode(unsigned int id, const unsigned char* data, unsigned char len)`

解码接收到的 CAN/RS485 数据帧。

- **参数**：
    - `id`：接收到的 CAN ID 或设备地址

    - `data`：数据指针（长度由 `len` 指定）
    - `len`：数据长度（通常为 1~8 字节）
- **说明**：此函数应在底层接收中断或线程中被调用，用于将原始数据送入 SDK 解析

`void set_send_rpdo_callback(ECSendDataCallback callback)`

设置 EtherCAT RPDO 发送回调（用于输出数据）。

- **参数**：
    - `callback`：函数指针，原型为 `bool(const unsigned char* data, int size)`
        - `data`：待发送的 RPDO 数据
        - `size`：数据总长度
        - 返回值：`true` 成功，`false` 失败
- **说明**：该回调将被 `get_pre_send_rpdo_data` 触发调用（若已设置）

`void set_send_rpdo_callback_ex(void* callback_impl)`

设置 `std::function` 包装的 EtherCAT 发送回调。

- **参数**：
    - `callback_impl`：指向 `std::function<bool(const unsigned char*, unsigned int)>` 的指针
- **注意**：生命周期管理由用户负责

`int get_pre_send_rpdo_data(unsigned char* data_ptr, int* io_size)`

获取待发送的 RPDO 数据（用于 EtherCAT 输出区）。

- **参数**：
    - `data_ptr`：输出缓冲区指针
        - 若为 `nullptr`，仅输出所需缓冲区大小
    - `io_size`：输入时表示缓冲区容量，输出时返回实际需要或已写入的字节数
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **说明**：若设置了 `set_send_rpdo_callback`，此函数会自动触发发送

`int set_tpdo_data_decode(const unsigned char* data_ptr, int data_size)`

解码从 EtherCAT 接收到的 TPDO 数据（输入数据）。

- **参数**:
  - `data_ptr`：TPDO 数据指针
  - `data_size`：数据总长度（字节）
- **返回值**:
  - `0`：成功
  - 其他：错误码
- **说明**：解码后可通过 `get_now_angle` 等接口获取最新状态

自由度与方向设置

`int set_dof_type(int dof_type)`

设置灵巧手自由度类型。

- **参数**:
  - `dof_type`：自由度类型，取值为 `LAC_DOF_6` 或 `LAC_DOF_15`
- **返回值**:
  - `0`：成功
  - `LER_PARAMETER`：参数无效
- **默认值**：`LAC_DOF_6`

`int get_dof(int* total, int* active)`

获取当前灵巧手的自由度信息。

- **参数**:
  - `total`：输出参数，返回总关节数（包括被动关节）
  - `active`：输出参数，返回主动电机数量
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int set_hand_direction(int dir)`

设置手的方向（影响运动映射逻辑）。

- **参数**:
  - `dir`：方向，取值为 `LDR_HAND_RIGHT` 或 `LDR_HAND_LEFT`

- **返回值**:
  - `0`：成功
  - `LER_PARAMETER`：参数错误

`int get_hand_direction(int* dir)`

获取当前手的方向。

- **参数**:
  - `dir`：输出参数，接收方向值（`LDR_HAND_RIGHT` 或 `LDR_HAND_LEFT`）
- **返回值**:
  - `0`：成功
  - 其他：错误码

电机控制接口

`int set_control_mode(int id, int mode = LCM_POSITION)`

设置电机控制模式。

- **参数**:
  - `id`：电机 ID，取值范围 `[0, DOF]`
    - `id = 0`：广播模式，设置所有电机
    - `id ≥ 1`：设置指定电机
  - `mode`：控制模式，取值参考 `LCM_*` 枚举
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_control_mode(int id, int* mode)`

获取指定电机的控制模式。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `mode`：输出参数，接收当前控制模式
- **返回值**:
  - `0`：成功
  - `LER_PARAMETER`：ID 超出范围

- 其他：错误码

`int set_enable(int id, int enable)`

使能或禁用电机驱动。

- **参数**：
  - `id`：电机 ID，取值范围 `[0, DOF]`
    - `id = 0`：广播设置所有电机
    - `id ≥ 1`：设置指定电机
  - `enable`：使能标志，`1` 表示使能，`0` 表示禁用
- **返回值**：
  - `0`：成功
  - 其他：错误码

`int get_enable(int id, int* enable)`

获取电机当前使能状态。

- **参数**：
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `enable`：输出参数，`1` 表示已使能，`0` 表示已禁用
- **返回值**：
  - `0`：成功
  - 其他：错误码

`int get_position_reached(int id, int* reached)`

获取电机是否已到达目标位置。

- **参数**：
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `reached`：输出参数，`1` 表示已到位，`0` 表示仍在运动
- **返回值**：
  - `0`：成功
  - 其他：错误码

`int set_clear_alarm(int id = 0)`

清除电机报警状态。

- **参数**:
  - `id`：电机 ID，取值范围 `[0, DOF]`
    - `id = 0`：广播清除所有电机报警
    - `id ≥ 1`：清除指定电机报警
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_now_alarm(int id, int* alarm)`

获取电机当前报警状态码。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `alarm`：输出参数，接收报警码（具体含义参考硬件手册）
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int home_motors(int id)`

启动电机回零操作。

- **参数**:
  - `id`：电机 ID，取值范围 `[0, DOF]`
    - `id = 0`：所有电机回零
    - `id ≥ 1`：指定电机回零
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_limit_target_angle(int id, float* max_angle, float* min_angle)`

获取电机目标角度的上下限。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`

    - `max_angle`：输出参数，最大允许角度（单位：°）
    - `min_angle`：输出参数，最小允许角度（单位：°）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_target_angle(int id, float angle)`

设置电机目标角度。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机
    - `angle`：目标角度，单位为 °（角度制）
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **注意**：实际可达范围由机械结构决定

`int get_target_angle(int id, float* angle)`

获取最近一次设置的目标角度。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `angle`：输出参数，接收目标角度值（单位：°）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_target_position(int id, int position)`

设置电机目标位置（行程当量）。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机

    - `position`：目标位置，单位为“当量”，取值范围 `[0, 10000]`
        - `0` 表示起始位
        - `10000` 表示满行程
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **注意**：实际行程由机械结构和校准决定

`int get_target_position(int id, int* position)`

获取最近一次设置的目标位置（行程当量）。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `position`：输出参数，接收目标位置值（单位：当量，范围 `[0, 10000]`）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_velocity(int id, float velocity)`

设置电机目标速度（默认使用角度速度单位）。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机
    - `velocity`：目标速度，单位为 °/s
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **注意**：实际速度受硬件限制；推荐使用 `set_angular_velocity` 以明确语义

`int get_velocity(int id, float* velocity)`

获取最近一次设置的目标速度（默认返回角度速度）。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`

    - `velocity`：输出参数，接收目标速度值（单位：°/s）
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **注意**：推荐使用 `get_angular_velocity` 以明确语义

`int set_angular_velocity(int id, float velocity)`

设置电机目标速度（单位：°/s）。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机
    - `velocity`：目标角速度，单位为 °/s
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **说明**：此接口与 `set_velocity` 功能一致，但命名更清晰，建议优先使用

`int get_angular_velocity(int id, float* velocity)`

获取最近一次设置的目标速度（单位：°/s）。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `velocity`：输出参数，接收目标角速度值（单位：°/s）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_position_velocity(int id, int velocity)`

设置电机目标速度（行程当量速度）。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机

    - `velocity`：目标速度，单位为 **当量/秒**（即每秒移动的当量数）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_position_velocity(int id, int* velocity)`

获取最近一次设置的目标速度（行程当量速度）。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `velocity`：输出参数，接收目标速度值（单位：当量/秒）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_max_current(int id, int current)`

设置电机最大允许电流。

- **参数**：
    - `id`：电机 ID，取值范围 `[0, DOF]`
        - `id = 0`：广播设置所有电机
        - `id ≥ 1`：设置指定电机
    - `current`：最大电流限制，单位为 mA
- **返回值**：
    - `0`：成功
    - 其他：错误码
- **注意**：用于过流保护和力矩控制上限

`int get_max_current(int id, int* current)`

获取最近一次设置的最大电流值。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `current`：输出参数，接收最大电流值（单位：mA）
- **返回值**：
    - `0`：成功

- 其他：错误码

`int move_motors(int id = 0)`

驱动电机执行已设置的运动指令。

- **参数**：
  - `id`：电机 ID，取值范围 `[0, DOF]`
    - `id = 0`：所有电机开始运动
    - `id ≥ 1`：指定电机开始运动
- **返回值**：
  - `0`：成功
  - 其他：错误码
- **说明**：需先调用 `set_target_angle` 等设置目标值

状态与反馈接口

`int get_now_status(int id, int* status)`

获取电机当前运行状态。

- **参数**：
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `status`：输出参数，接收运行状态码（参考 `LST_*` 枚举）
- **返回值**：
  - `0`：成功
  - 其他：错误码

`int get_now_angle(int id, float* angle)`

获取电机当前角度位置。

- **参数**：
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `angle`：输出参数，接收当前角度（单位：°）
- **返回值**：
  - `0`：成功
  - 其他：错误码

`int get_now_position(int id, int* position)`

获取电机当前行程位置（当量）。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `position`：输出参数，接收当前行程位置（单位：当量，范围 `[0, 10000]`）
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_now_velocity(int id, float* velocity)`

获取电机当前实际速度。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `velocity`：输出参数，接收当前速度（单位：°/s）
- **返回值**:
  - `0`：成功
  - 其他：错误码
- **注意**：此接口返回角度速度，推荐使用 `get_now_angular_velocity` 以明确语义

`int get_now_angular_velocity(int id, float* velocity)`

获取电机当前实际速度（单位：°/s）。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`
  - `velocity`：输出参数，接收当前角速度（单位：°/s）
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_now_position_velocity(int id, int* velocity)`

获取电机当前实际速度（当量速度）。

- **参数**:
  - `id`：电机 ID，取值范围 `[1, DOF]`

    - `velocity`：输出参数，接收当前速度（单位：当量/秒）
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_now_current(int id, int* current)`

获取电机当前电流。

- **参数**：
    - `id`：电机 ID，取值范围 `[1, DOF]`
    - `current`：输出参数，接收当前电流值（单位：mA）
- **返回值**：
    - `0`：成功
    - 其他：错误码

触觉传感器接口

`int get_finger_sensor_pos(int sensor_id, double* x_values, double* y_values, int* io_count)`

获取触觉传感器分布坐标（归一化 `[0.0, 1.0]`）。

- **参数**：
    - `sensor_id`：传感器 ID，取值参考 `LSS_*` 枚举
    - `x_values`：输出参数，X 坐标数组指针（可为 `nullptr` 仅获取数量）
    - `y_values`：输出参数，Y 坐标数组指针
    - `io_count`：输入时表示缓冲区大小，输出时返回实际元素数量
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_finger_pressure(int sensor_id, double* pressure_list, int* io_count)`

获取传感器压力值（范围 `[0.0, 1.0]`）。

- **参数**：
    - `sensor_id`：传感器 ID
    - `pressure_list`：输出参数，压力值数组指针（可为 `nullptr` 仅获取数量）

    - `io_count`：输入为缓冲区大小，输出为实际数量
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int set_finger_pressure_reset(int sensor_id)`

重置传感器基准值（差值从此刻开始计算）。

- **参数**：
    - `sensor_id`：传感器 ID
        - `sensor_id = 0`：重置所有传感器
        - `sensor_id > 0`：仅重置指定传感器
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_finger_normal_force(int sensor_id, double* normal_force)`

获取法向力（垂直于表面）。

- **参数**：
    - `sensor_id`：传感器 ID
    - `normal_force`：输出参数，接收法向力
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_finger_tangential_force(int sensor_id, double* tangential_force)`

获取切向力（平行于表面）。

- **参数**：
    - `sensor_id`：传感器 ID
    - `tangential_force`：输出参数，接收切向力
- **返回值**：
    - `0`：成功
    - 其他：错误码

`int get_finger_force_direction(int sensor_id, double* force_direction)`

获取受力方向。

- **参数**:
  - `sensor_id`：传感器 ID
  - `force_direction`：输出参数，接收方向角度（范围 `[0, 360]`，单位：°）
- **返回值**:
  - `0`：成功
  - 其他：错误码

`int get_finger_proximity(int sensor_id, double* proximity)`

获取接近程度（如接近传感器）。

- **参数**:
  - `sensor_id`：传感器 ID
  - `proximity`：输出参数，接收接近程度（范围 `[0.0, 1.0]`）
- **返回值**:
  - `0`：成功
  - 其他：错误码

日志系统

`void log_on(bool on = false, int maxsize = 10000)`

开启或关闭日志打印功能。

- **参数**:
  - `on`： `true` 开启日志， `false` 关闭（默认）
  - `maxsize`：最大缓存日志条数，防止内存溢出

`int log_send(int* cmd, int count)`

设置需要打印的发送指令地址过滤列表。

- **参数**:
  - `cmd`：指令地址数组指针
    - `nullptr` 表示打印所有发送数据
  - `count`：数组元素个数

- **返回值**:
  - `0`：成功
  - 其他：错误码

`int log_recv(int* cmd, int count)`

设置需要打印的接收指令地址过滤列表。

- **参数**:
  - `cmd`：指令地址数组指针
    - `nullptr` 表示打印所有接收数据
  - `count`：数组元素个数
- **返回值**:
  - `0`：成功
  - 其他：错误码

`void log_reset(bool send = true, bool recv = true)`

清空日志过滤地址列表。

- **参数**:
  - `send`：是否清空发送过滤列表
  - `recv`：是否清空接收过滤列表

`int log_save(const char* file_name)`

将当前日志保存到文件。

- **参数**:
  - `file_name`：文件路径
- **返回值**:
  - `0`：成功
  - `LER_INVALID_PATH`：路径无效
  - `LER_LOG_SAVE_FAIL`：保存失败

`void log_clear()`

清空当前内存中的日志记录。

`void set_log_callback(LogAddCallback callback)`

设置日志回调函数（实时接收日志字符串）。

- **参数**：
  - `callback`：函数指针，原型为 `void(const char*)`
    - 输入参数为日志字符串（以 `\n` 结尾）
- **示例**：

```
lhp_lib_->set_log_callback([](const char* msg) {
    printf("[SDK_LOG] %s", msg);
});
```

使用流程示例（EtherCAT 通讯）

```
#include "LHandProLib.h"
#include <iostream>
#include <chrono>
#include <thread>
#include <memory>
#include <functional>
#include "EthercatMaster.h" // EtherCAT主站库

int main() {
  /****** 初始化 ******/
  // 初始化LHandProLib对象
  auto lhp_lib_ = std::make_shared<lhplib::LHandProLib>();
  // 初始化EtherCAT主站对象
  auto ec_master_ = std::make_shared<EthercatMaster>();

  // 初始化EtherCAT主站
  std::vector<std::string> names = ec_master_->scanNetworkInterfaces();
  int channel_index = 0; // 根据扫描到的网口列表选择正确的通道
  if (!ec_master_->init(channel_index)) {
    std::cout << "初始化失败" << std::endl;
    return -1;
  }
  std::cout << "连接成功" << std::endl;

  // EtherCAT主站进入OP
  if (!ec_master_->start()) {
    std::cout << "启动设备失败" << std::endl;
    return -1;
  }

  // EtherCAT主站启动后台数据交换通信（非阻塞）
  ec_master_->run();
```

```cpp
/****** 设置发送RPDO回调以及TPDO的处理 ******/
// EtherCAT的发送函数
auto send_func = [ec_master_](const unsigned char* data, unsigned int size) {
  return ec_master_->setOutputs(data, size);
};
// 使用std::function包装
std::function<bool(const unsigned char*, unsigned int)> func = send_func;

// 设置EtherCAT发送数据的回调
lhp_lib_->set_send_rpdo_callback_ex(&func);

// 建立监控线程，通过接口解析TPDO数据
std::atomic<bool> stop{false};
std::thread t_monitor([=, &stop]() {
  while (!stop) {
    // 循环获取TPDO的数据，并交由SDK进行解析
    int input_size = ec_master_->getInputSize();
    std::vector<uint8_t> in_buffer(input_size);
    if (ec_master_->getInputs(in_buffer.data(), input_size)) {
      lhp_lib_->set_tpdo_data_decode(in_buffer.data(), input_size);
    }
    std::this_thread::sleep_for(std::chrono::milliseconds(10));
  }
});

// 初始化库
if (lhp_lib_->initial(lhplib::LCN_ECAT) != lhplib::LER_NONE) {
  std::cerr << "LHandProLib 初始化失败！" << std::endl;
  return -1;
}


/****** 运动的接口调用 ******/
// 设置控制模式
lhp_lib_->set_control_mode(0, lhplib::LCM_POSITION);
// 全部使能
lhp_lib_->set_enable(0, 1);


// 获取当前自由度
int dof_total = 0, dof_active = 0;
lhp_lib_->get_dof(&dof_total, &dof_active);

// 每个关节的目标角度
std::vector<float> target_angles = {30.0f, 10.0f, 60.0f, 60.0f, 60.0f, 60.0f};

// 设置运动参数
for (int i = 0; i < dof_active; ++i) {
  lhp_lib_->set_target_angle(i + 1, target_angles[i]);  // 设置目标角度
  lhp_lib_->set_angular_velocity(i + 1, 200);           // 设置速度（200°/s）
  lhp_lib_->set_max_current(i + 1, 800);                // 设置最大电流限制（800mA）
}
```

```cpp
    // 驱动所有电机运动
    lhp_lib_->move_motors();
    // 等待运动结束
    std::this_thread::sleep_for(std::chrono::milliseconds(1000));

    // 获取当前角度
    for (int i = 0; i < dof_active; ++i) {
      float angle = 0;
      if (lhp_lib_->get_now_angle(i + 1, &angle) == lhplib::LER_NONE) {
        std::cout << "当前关节: " << i + 1 << " 角度: " << angle << "°" << std::endl;
      }
    }

    /****** 传感器的接口调用 ******/
    // 读取大拇指指尖压力分布
    int count = 0;
    lhp_lib_->get_finger_pressure(lhplib::LSS_FINGER_1_1, nullptr, &count);
    std::vector<float> pressures(count);
    lhp_lib_->get_finger_pressure(lhplib::LSS_FINGER_1_1, pressures.data(), &count);

    for (int i = 0; i < count; ++i) {
        std::cout << "传感器: " << i << " 压力: " << pressures[i] << std::endl;
    }

    // 读取法向力
    float normal_force;
    lhp_lib_->get_finger_normal_force(lhplib::LSS_FINGER_1_1, &normal_force);
    std::cout << "传感器: " << lhplib::LSS_FINGER_1_1 << " 法向力: " << normal_force <<
std::endl;

    // 读取切向力
    float tangential_force;
    lhp_lib_->get_finger_tangential_force(lhplib::LSS_FINGER_1_1, &tangential_force);
    std::cout << "传感器: " << lhplib::LSS_FINGER_1_1 << " 切向力: " << tangential_force
<< std::endl;

    // 读取切向力方向
    float force_direction;
    lhp_lib_->get_finger_force_direction(lhplib::LSS_FINGER_1_1, &force_direction);
    std::cout << "传感器: " << lhplib::LSS_FINGER_1_1 << " 切向力方向: " <<
force_direction << std::endl;

    // 读取接近程度
    float proximity;
    lhp_lib_->get_finger_proximity(lhplib::LSS_FINGER_1_1, &proximity);
    std::cout << "传感器: " << lhplib::LSS_FINGER_1_1 << " 接近程度: " << proximity <<
std::endl;

    // 关闭
    lhp_lib_->close();

    return 0;
}
```

其他示例

详见SDK开发包中的各通讯版本的示例工程。

> LHandProLib-API-share-LHandProLib-examples

```
LHandProLib-API
 ├── bin
 │   ├── LHandProLib.dll
 │   ├── LHandProLib.pdb
 │   ├── LHandProLibd.dll
 │   ├── LHandProLibd.pdb
 │   ├── LHandProLib_EtherCAT_Test.exe
 │   └── LHandProLib_PCAN_Test.exe
 ├── include
 │   └── LHandProLib
 │       └── LHandProLib.h
 ├── lib
 │   ├── LHandProLib.lib
 │   └── LHandProLibd.lib
 └── share
     └── LHandProLib
         └── examples
             ├── EtherCAT
             └── PCAN
```

常见问题

Q: 为什么 `initial()` 返回 `LER_KEY_FUNC_UNINIT`?

A: 未设置 `set_send_data_callback` 或 `set_recv_data_decode` 回调函数。

Q: 如何查看通讯数据?

A: 使用 `log_on(true)` 并调用 `log_send(nullptr, 0)` 和 `log_recv(nullptr, 0)` 打印所有数据。

Q: 传感器数据不更新?

A: 确保已周期性调用 `set_recv_data_decode`（CAN）或 `set_tpdo_data_decode`（EtherCAT）。

版本更新记录

- **v1.0**：初始版本，支持 EtherCAT，6/15 DOF，触觉传感器读取，日志系统。
- **v1.1**：新增当量控制接口，优化EtherCAT示例程序。

版权声明： © 2025 深圳市灵巧驱控技术有限公司。保留所有权利。
用户协议： 使用本软件即表示您同意用户协议。

文档版本：v1.1 | 更新日期：2025年9月8日